

SPRAWOZDANIE Z PROJEKTU SYSTEMÓW OPERACYJNYCH

FolderSyncDaemon

Demon synchronizacji katalogów (Linux, C, POSIX)

Wykonawcy: Aleksandr Kozłowski

Mykhailo Pashuk

Aleksandra Kruhlova

Przedmiot: Systemy Operacyjne

Prowadzący: mgr Damian Hańko

Data: 2025 / 2026

Spis treści

1. Treść zadania i wymagania
2. Struktura projektu
3. Opis najważniejszych modułów
 - 3.1 main.c — punkt wejścia i główna pętla
 - 3.2 daemon.c / daemon.h — daemonizacja
 - 3.3 signals.c / signals.h — obsługa sygnałów
 - 3.4 sync.c / sync.h — synchronizacja katalogów
 - 3.5 copy.c / copy.h — kopiowanie plików
4. Opis najistotniejszych algorytmów
 - 4.1 Algorytm daemonizacji (double-fork)
 - 4.2 Algorytm synchronizacji dwuprzebiegowej
 - 4.3 Algorytm kopiowania (read/write vs mmap/write)
 - 4.4 Algorytm usuwania rekurencyjnego
5. Zmienne globalne i interfejsy publiczne
6. Kod programu z komentarzami
 - 6.1 main.c
 - 6.2 daemon.c
 - 6.3 signals.c
 - 6.4 sync.c
 - 6.5 copy.c
7. Instrukcja kompilacji i uruchomienia

1. Treść zadania i wymagania

Projekt realizuje program demona systemu Linux służącego do jednostronnej synchronizacji dwóch katalogów. Poniżej przedstawiono pełną treść zadania oraz punktację.

[12 pkt.] Program który otrzymuje co najmniej dwa argumenty: ścieżkę źródłową oraz ścieżkę docelową. Jeżeli któraś ze ścieżek nie jest katalogiem program powraca natychmiast z komunikatem błędu. W przeciwnym wypadku staje się demonem. Demon wykonuje następujące czynności: śpi przez pięć minut (czas spania można zmieniać przy pomocy dodatkowego opcjonalnego argumentu), po czym po obudzeniu się porównuje katalog źródłowy z katalogiem docelowym. Pozycje które nie są zwykłymi plikami są ignorowane (np. katalogi i dowiązania symboliczne). Jeżeli demon (a) napotka na nowy plik w katalogu źródłowym, i tego pliku brak w katalogu docelowym lub (b) plik w katalogu źródłowym ma późniejszą datę ostatniej modyfikacji — demon wykonuje kopię pliku z katalogu źródłowego do katalogu docelowego, ustawiając w katalogu docelowym datę modyfikacji tak aby przy kolejnym obudzeniu nie trzeba było wykonać kopii (chyba że plik w katalogu źródłowym zostanie ponownie zmieniony). Jeżeli zaś odnajdzie plik w katalogu docelowym, którego nie ma w katalogu źródłowym to usuwa ten plik z katalogu docelowego. Możliwe jest również natychmiastowe obudzenie się demona poprzez wysłanie mu sygnału SIGUSR1. Wyczerpująca informacja o każdej akcji typu uśpienie/obudzenie się demona (naturalne lub w wyniku sygnału), wykonanie kopii lub usunięcie pliku jest przesłana do logu systemowego.

Rozszerzenia:

[+10 pkt.] Opcja -R — rekurencyjna synchronizacja katalogów. Pozycje będące katalogami nie są ignorowane. Jeżeli demon stwierdzi w katalogu docelowym podkatalog, którego brak w katalogu źródłowym, powinien usunąć go wraz z zawartością.

[+12 pkt.] Próg mmap (-t) — w zależności od rozmiaru pliku dla małych plików wykonywane jest kopiowanie przy pomocy read/write, a dla dużych przy pomocy mmap/write (plik źródłowy zostaje zamapowany w całości w pamięci). Próg dzielący pliki małe od dużych może być przekazywany jako opcjonalny argument.

Ograniczenia narzucone przez prowadzącego:

- Wszelkie operacje na plikach i tworzenie demona należy wykonywać przy pomocy API Linuksa (POSIX) — zakazana jest funkcja daemon().
- Kopiowanie za każdym obudzeniem całego drzewa katalogów (bez porównywania mtime) zostanie potraktowane jako poważny błąd.
- Zakazane jest przerzucanie części zadań na shell systemowy (funkcja system()).

2. Struktura projektu

Projekt składa się z pięciu modułów (par .c/.h) i pliku Makefile. Każdy moduł realizuje dokładnie jedną odpowiedzialność zgodnie z zasadą rozdziału zagadnień (Separation of Concerns).

Plik	Rola
main.c	Parsowanie argumentów, weryfikacja ścieżek, uruchomienie demonu, główna pętla
daemon.c/h	Daemonizacja procesu (double-fork, setsid, umask, chdir, /dev/null)
signals.c/h	Instalacja handlera SIGUSR1, flaga wakeup_requested
sync.c/h	Dwuprzebiegowa synchronizacja katalogów, usuwanie rekurencyjne
copy.c/h	Kopiowanie pliku metodą read/write lub mmap/write, przywracanie mtime
Makefile	Kompilacja całego projektu (gcc -Wall -Wextra -std=c11)

3. Opis najważniejszych modułów

3.1 main.c — punkt wejścia i główna pętla

Moduł main.c jest punktem wejścia programu. Realizuje pięć zadań:

- Parsowanie argumentów — przy użyciu getopt() przetwarza flagi -R (rekurencja) i -t bajty (próg mmap) oraz argumenty pozycyjne: ścieżkę źródłową, docelową i opcjonalny interwał snu.
- Weryfikacja ścieżek — przed daemonizacją sprawdza przez stat(), czy obie ścieżki istnieją i są katalogami; błędy są wypisywane na stderr.
- Konwersja ścieżek do bezwzględnych — realpath() zamienia ścieżki względne na bezwzględne, ponieważ po daemonizacji katalog roboczy zmienia się na / (patrz daemon.c).
- Uruchomienie demona — kolejno: daemonize(), setup_signals(), openlog().
- Główna pętla — w nieskończoność wywołuje synchronize_folders(), następnie śpi przez zadany czas; sen jest przerywany przez SIGUSR1.

3.2 daemon.c / daemon.h — daemonizacja

Funkcja daemonize() przekształca bieżący proces w demona zgodnie ze standardem POSIX, bez użycia zakazanej funkcji daemon(3). Wykonuje sześć kroków:

Krok	Cel
fork() #1	Rodzic kończy działanie — shell odzyskuje kontrolę i wyświetla prompt.
setsid()	Dziecko tworzy nową sesję, odłączając się od terminala sterującego.
fork() #2	Gwarancja, że demon nie jest liderem sesji i nie może ponownie przejąć terminala przez open() na /dev/ttyX.
umask(0)	Wyłączenie domyślnego maskowania uprawnień — demon sam jawnie ustawia prawa w open()/mkdir().
chdir("/")	Zwolnienie bieżącego systemu plików (nie może być odmontowany, jeśli demon trzyma w nim CWD).
stdin/out/err → /dev/null	Wszystkie trzy standardowe deskryptory przekierowane na /dev/null zapobiegają przypadkowemu I/O na terminal.

3.3 signals.c / signals.h — obsługa sygnałów

Moduł udostępnia mechanizm natychmiastowego budzenia demona przez sygnał SIGUSR1. Składa się z dwóch elementów:

- volatile sig_atomic_t wakeup_requested — flaga ustawiana przez handler; deklaracja volatile sig_atomic_t gwarantuje atomowy odczyt/zapis bez wyścigu danych (data race) z kodem nieobsługi sygnałów.
- setup_signals() — instaluje handler przez sigaction() (preferowane nad signal()). Maska sygnałów handlera jest pusta — handler może zostać przerwany jedynie przez sygnały nieblokowane.

Główna pętla w main.c sprawdza flagę po każdym zakończonym sleep() i zeruje ją przed kolejnym cyklem synchronizacji.

3.4 sync.c / sync.h — synchronizacja katalogów

Funkcja `synchronize_folders(source, target, recursive, threshold)` realizuje kompletną logikę synchronizacji w dwóch przebiegach:

- Przebieg 1 (`source` → `target`): iteracja po źródle przez `opendir()/readdir()`. Dla zwykłych plików: kopiuj jeśli brak w celu lub `mtime` źródła jest nowszy. Dla katalogów (tylko z `-R`): utwórz `mkdir()` jeśli brak, potem rekurencja.
- Przebieg 2 (usuwanie z `target`): iteracja po celu; jeśli pozycja nie istnieje w źródle — usuń (`unlink()` dla pliku lub `remove_recursive()` dla katalogu).

Dowiązania symboliczne, urządzenia i inne typy plików są świadomie ignorowane w obu przebiegach (zgodnie z treścią zadania).

3.5 copy.c / copy.h — kopiowanie plików

Moduł eksportuje jedną funkcję publiczną `copy_file(src, dst, st, threshold)` i dwie statyczne funkcje wewnętrzne:

- `copy_rw(src_fd, dst_fd, src_path)` — kopiowanie przez bufor 4 KB (`read()/write()`). Wewnętrzna pętla zapisu obsługuje przypadek, gdy `write()` zapisuje mniej bajtów niż żądano (np. sieciowe systemy plików).
- `copy_mmap(src_fd, dst_fd, size, src_path)` — mapuje cały plik źródłowy przez `mmap(PROT_READ, MAP_SHARED)`, następnie zapisuje do celu przez `write()`. W razie niepowodzenia `mmap` automatycznie przełącza się na `copy_rw()`.
- `copy_file()` — otwiera deskryptory, wybiera metodę na podstawie progu, a po skopiowaniu przywraca `mtime` przez `utime()` — co zapobiega ponownemu kopiowaniu niezmienionego pliku przy kolejnym obudzeniu.

4. Opis najistotniejszych algorytmów

4.1 Algorytm daemonizacji (double-fork)

Poniżej przedstawiono przepływ sterowania w funkcji `daemonize()`:

```
// double-fork daemonization pseudocode
Proces startowy
|
+-- fork() #1
| |
| [RODZIC] exit(0) <-- shell widzi koniec, zwraca prompt
|
+-- [DZIECKO #1] setsid() <-- nowa sesja, brak terminala
|
+-- fork() #2
| |
| [DZIECKO #1] exit(0) <-- pośredni rodzic kończy
|
+-- [DZIECKO #2 = DEMON]
    umask(0)
    chdir("/")
    dup2(/dev/null, 0/1/2)
    --> openlog() + główna pętla
```

Kluczowe jest, że demon (dziecko #2) nie jest liderem sesji — nie może przejąć terminala sterującego nawet jeśli otworzy urządzenie `/dev/ttyX`. Jest to standardowa technika opisana w książce W. Richarda Stevensa *Advanced Programming in the UNIX Environment*.

4.2 Algorytm synchronizacji dwuprzebiegowej

Synchronizacja wymaga dwóch niezależnych przebiegów, ponieważ każdy realizuje inny kierunek przepływu informacji:

```
// synchronize_folders pseudocode
PRZEBIEG 1 – source → target:
FOR każda pozycja e w opendir(source):
    src = source/e
    dst = target/e
    IF S_ISREG(src):
        IF !exists(dst) OR mtime(src) > mtime(dst):
            copy_file(src, dst) // kopiuj (z przywróceniem mtime)
    ELSE IF S_ISDIR(src) AND recursive:
        IF !exists(dst): mkdir(dst)
        synchronize_folders(src, dst) // rekurencja
PRZEBIEG 2 – usuwanie z target:
FOR każda pozycja e w opendir(target):
    src = source/e
    dst = target/e
    IF !exists(src):
        IF S_ISREG(dst): unlink(dst)
        IF S_ISDIR(dst) AND recursive: remove_recursive(dst)
```

Poprawność porównania `mtime` zależy od tego, że `copy_file()` po skopiowaniu przywraca czas modyfikacji przez `utime()`. Bez tego każde obudzenie demona traktowałoby wszystkie pliki jako zmienione.

4.3 Algorytm kopiowania (read/write vs mmap/write)

Wybór metody zależy od rozmiaru pliku względem progu threshold:

```
// copy_file pseudocode
copy_file(src, dst, st, threshold):
    open src O_RDONLY
    open dst O_WRONLY|O_CREAT|O_TRUNC
    IF st.st_size >= threshold: // duże pliki
        mapped = mmap(NULL, size, PROT_READ, MAP_SHARED, src_fd, 0)
        IF mapped == MAP_FAILED:
            fallback → copy_rw() // automatyczny fallback
    ELSE:
        ptr = mapped
        WHILE remaining > 0:
            w = write(dst_fd, ptr, remaining)
            IF w < 0 AND errno == EINTR: continue // obsługa sygnałów
            ptr += w; remaining -= w
        munmap(mapped, size)
    ELSE: // małe pliki
        WHILE (n = read(src_fd, buf, 4096)) > 0:
            written = 0
            WHILE written < n:
                w = write(dst_fd, buf+written, n-written)
                written += w
        utime(dst, {atime=src.atime, mtime=src.mtime}) // przywróć mtime
```

4.4 Algorytm usuwania rekurencyjnego (remove_recursive)

Funkcja `remove_recursive()` realizuje odpowiednik polecenia `rm -rf` przez post-order DFS (najpierw usuń dzieci, potem katalog-rodzic):

```
// remove_recursive pseudocode
remove_recursive(path):
    lstat(path, &st)
    IF S_ISREG OR S_ISLNK: unlink(path); return
    IF NOT S_ISDIR: return // inne typy – pomijamy
    dir = opendir(path)
    FOR każde dziecko e w dir (pomijamy . i ..):
        remove_recursive(path + "/" + e) // rekurencja w głąb
    closedir(dir)
    rmdir(path) // katalog jest teraz pusty
```


5. Zmienne globalne i interfejsy publiczne

Program stosuje minimalną liczbę zmiennych globalnych — tylko te wymagane przez mechanizm obsługi sygnałów:

Nazwa	Typ	Plik	Opis
wakeup_requested	volatile sig_atomic_t	signals.c	Flaga ustawiana przez handler SIGUSR1. Odczytywana i zerowana w main.c po każdym przerwaniu sleep().

Publiczne funkcje modułów:

Sygnatura	Moduł	Opis
void daemonize(void)	daemon.c	Przekształca proces w demona (double-fork, setsid, umask, chdir, /dev/null).
void setup_signals(void)	signals.c	Instaluje handler SIGUSR1 przez sigaction().
void synchronize_folders([[]src, dst, [] recursive, threshold)	sync.c	Dwuprzebiegowa synchronizacja katalogów. Kopiuje nowe/zmienione pliki, usuwa nadmiarowe.
void copy_file([[]src, dst, [] st, threshold)	copy.c	Kopiuje plik metodą read/write lub mmap/write zależnie od progu. Przywraca mtime.

6. Kod programu z komentarzami

Poniżej zamieszczono pełny kod źródłowy wszystkich plików .c. Komentarze w kodzie opisują logikę poszczególnych sekcji.

6.1 main.c

```
// main.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h> /* getopt(), sleep() */
#include <sys/stat.h>
#include <syslog.h>
#include <stdbool.h>
#include <string.h>
#include <limits.h> /* PATH_MAX – potrzebne do realpath() */
#include "daemon.h"
#include "signals.h"
#include "sync.h"
#define DEFAULT_SLEEP_SECS 300 /* 5 minut */
#define DEFAULT_THRESHOLD (100*1024) /* 100 KB */
static void usage(const char *prog) { /* ... wypisuje pomoc ... */ }
int main(int argc, char *argv[]) {
    bool recursive = false;
    off_t threshold = DEFAULT_THRESHOLD;
    /* Parsowanie opcji -R i -t <bajty> */
    int opt;
    while ((opt = getopt(argc, argv, "Rt:h")) != -1) { ... }
    /* Argumenty pozycyjne: source, target, [sleep_sec] */
    char *source = argv[optind];
    char *target = argv[optind + 1];
    int sleep_sec = (optind+2 < argc) ? atoi(argv[optind+2]) : DEFAULT_SLEEP_SECS;
    /* Weryfikacja: stat() sprawdza czy ścieżki są katalogami */
    struct stat st;
    if (stat(source, &st) != 0 || !S_ISDIR(st.st_mode)) { error; }
    if (stat(target, &st) != 0 || !S_ISDIR(st.st_mode)) { error; }
    /* Konwersja na ścieżki bezwzględne – PRZED daemonize! */
    /* (daemonize wykonuje chdir("/"), więc względne ścieżki przestają działać) */
    char abs_source[PATH_MAX], abs_target[PATH_MAX];
    if (!realpath(source, abs_source)) { perror("realpath"); return EXIT_FAILURE; }
    if (!realpath(target, abs_target)) { perror("realpath"); return EXIT_FAILURE; }
    daemonize(); /* stajemy się demonem */
    setup_signals(); /* instalujemy handler SIGUSR1 */
    openlog("FolderSyncDaemon", LOG_PID|LOG_NDELAY, LOG_DAEMON);
    syslog(LOG_INFO, "Demon uruchomiony. Źródło: %s ...", abs_source, ...);
    /* Główna pętla */
    while (1) {
        synchronize_folders(abs_source, abs_target, recursive, threshold);
        /* sleep() jest przerywany przez SIGUSR1 */
        int remaining = sleep_sec;
        while (remaining > 0 && !wakeup_requested)
            remaining = (int)sleep((unsigned)remaining);
        if (wakeup_requested) { wakeup_requested = 0; /* zaloguj */ }
    }
    closelog();
    return EXIT_SUCCESS;
}
```

6.2 daemon.c

```
// daemon.c
#include "daemon.h"
#include <unistd.h> /* fork(), setsid(), chdir(), dup2() */
#include <sys/stat.h> /* umask() */
#include <fcntl.h> /* open(), O_RDWR */
#include <stdlib.h> /* exit() */
void daemonize(void) {
    /* Krok 1: pierwsze rozwidlenie */
    /* Rodzic kończy → shell odzyskuje kontrolę */
    pid_t pid = fork();
    if (pid < 0) { perror("fork (1)"); exit(EXIT_FAILURE); }
    if (pid > 0) exit(EXIT_SUCCESS);
    /* Krok 2: nowa sesja – odłączenie od terminala */
    if (setsid() < 0) { perror("setsid"); exit(EXIT_FAILURE); }
    /* Krok 3: drugie rozwidlenie – demon NIE jest liderem sesji */
    pid = fork();
    if (pid < 0) { perror("fork (2)"); exit(EXIT_FAILURE); }
    if (pid > 0) exit(EXIT_SUCCESS);
    /* Krok 4: maska uprawnień – brak ukrytych ograniczeń */
    umask(0);
    /* Krok 5: katalog roboczy → / (zwalniamy bieżący fs) */
    if (chdir("/") < 0) { perror("chdir"); exit(EXIT_FAILURE); }
    /* Krok 6: stdin/stdout/stderr → /dev/null */
    int devnull = open("/dev/null", O_RDWR);
    if (devnull >= 0) {
        dup2(devnull, STDIN_FILENO);
        dup2(devnull, STDOUT_FILENO);
        dup2(devnull, STDERR_FILENO);
        if (devnull > STDERR_FILENO) close(devnull);
    }
}
```

6.3 signals.c

```
// signals.c
#include "signals.h"
#include <string.h>
#include <unistd.h>
/* Definicja flagi – extern w signals.h */
volatile sig_atomic_t wakeup_requested = 0;
/* Handler SIGUSR1 – ustawia flagę (async-signal-safe) */
static void sigusr1_handler(int sig) {
    (void)sig;
    wakeup_requested = 1;
}
void setup_signals(void) {
    struct sigaction sa;
    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = sigusr1_handler;
    sigemptyset(&sa.sa_mask);
    /* SA_RESTART nie jest ustawione – sleep() zostanie przerwany */
    sigaction(SIGUSR1, &sa, NULL);
}
```

6.4 sync.c

```
// sync.c – część 1: remove_recursive
#include "sync.h"
#include "copy.h"
#include <sys/stat.h>
#include <dirent.h> /* opendir, readdir, closedir */
#include <unistd.h> /* unlink, rmdir */
#include <string.h>
#include <limits.h> /* PATH_MAX */
#include <syslog.h>, <errno.h>, <stdio.h>
/* Usuwa path i całą zawartość (post-order DFS = rm -rf) */
static void remove_recursive(const char *path) {
    struct stat st;
    if (lstat(path, &st) != 0) return;
    if (S_ISREG(st.st_mode) || S_ISLNK(st.st_mode)) {
        unlink(path); return;
    }
    if (!S_ISDIR(st.st_mode)) return;
    DIR *dir = opendir(path);
    struct dirent *entry;
    char child[PATH_MAX];
    while ((entry = readdir(dir)) != NULL) {
        if (strcmp(entry->d_name, ".") == 0 ||
            strcmp(entry->d_name, "..") == 0) continue;
        snprintf(child, PATH_MAX, "%s/%s", path, entry->d_name);
        remove_recursive(child); /* rekurencja w głąb */
    }
    closedir(dir);
    rmdir(path); /* katalog pusty – usuń */
}
```

```
// sync.c – część 2: synchronize_folders
void synchronize_folders(const char *source, const char *target,
                        bool recursive, off_t threshold) {
    struct stat st_src, st_dst;
    char src_path[PATH_MAX], dst_path[PATH_MAX];
    /* — Przebieg 1: source → target — */
    DIR *dir = opendir(source);
    struct dirent *entry;
    while ((entry = readdir(dir)) != NULL) {
        snprintf(src_path, PATH_MAX, "%s/%s", source, entry->d_name);
        snprintf(dst_path, PATH_MAX, "%s/%s", target, entry->d_name);
        lstat(src_path, &st_src);
        if (S_ISREG(st_src.st_mode)) {
            /* kopiuj: brak w celu LUB nowszy mtime */
            if (lstat(dst_path, &st_dst) != 0 ||
                st_src.st_mtime > st_dst.st_mtime)
                copy_file(src_path, dst_path, &st_src, threshold);
        } else if (recursive && S_ISDIR(st_src.st_mode)) {
            if (lstat(dst_path, &st_dst) != 0) mkdir(dst_path, ...);
            synchronize_folders(src_path, dst_path, recursive, threshold);
        }
        /* symplinki, urządzenia itp. – ignorujemy */
    }
    closedir(dir);
    /* — Przebieg 2: usuwanie z target — */
    dir = opendir(target);
    while ((entry = readdir(dir)) != NULL) {
        snprintf(src_path, PATH_MAX, "%s/%s", source, entry->d_name);
        snprintf(dst_path, PATH_MAX, "%s/%s", target, entry->d_name);
        lstat(dst_path, &st_dst);
        if (lstat(src_path, &st_src) != 0) { /* brak w źródle */
            if (S_ISREG(st_dst.st_mode)) unlink(dst_path);
            else if (recursive && S_ISDIR(st_dst.st_mode))
                remove_recursive(dst_path);
        }
    }
    closedir(dir);
}
```

6.5 copy.c

```
// copy.c
#include "copy.h"
#include <fcntl.h> /* open */
#include <unistd.h> /* read, write, close */
#include <sys/mman.h> /* mmap, munmap */
#include <utime.h> /* utime, struct utimbuf */
#include <syslog.h>
#include <string.h>
#include <errno.h>
/* Kopiowanie przez bufor 4 KB (małe pliki lub fallback) */
static void copy_rw(int src_fd, int dst_fd, const char *src_path) {
    char buf[4096];
    ssize_t n;
    while ((n = read(src_fd, buf, sizeof(buf))) > 0) {
        ssize_t written = 0;
        while (written < n) {
            /* pętla write() – obsługa częściowych zapisów */
            ssize_t w = write(dst_fd, buf+written, (size_t)(n-written));
            if (w < 0) { syslog(LOG_ERR, ...); return; }
            written += w;
        }
    }
}
/* Kopiowanie przez mmap (duże pliki) */
static void copy_mmap(int src_fd, int dst_fd,
                     off_t size, const char *src_path) {
    void *mapped = mmap(NULL, size, PROT_READ, MAP_SHARED, src_fd, 0);
    if (mapped == MAP_FAILED) {
        /* Automatyczny fallback na read/write */
        syslog(LOG_WARNING, "mmap nieudane – przełączam na read/write");
        copy_rw(src_fd, dst_fd, src_path);
        return;
    }
    const char *ptr = mapped;
    size_t remaining = (size_t)size;
    while (remaining > 0) {
        ssize_t w = write(dst_fd, ptr, remaining);
        if (w < 0) {
            if (errno == EINTR) continue; /* przerwany przez sygnał */
            syslog(LOG_ERR, ...); break;
        }
        ptr += w; remaining -= w;
    }
    munmap(mapped, size);
}
/* Publiczny interfejs – wybiera metodę i przywraca mtime */
void copy_file(const char *src_path, const char *dst_path,
              const struct stat *st, off_t threshold) {
    int src_fd = open(src_path, O_RDONLY);
    int dst_fd = open(dst_path, O_WRONLY|O_CREAT|O_TRUNC, st->st_mode & 0777);
    if (st->st_size >= threshold)
        copy_mmap(src_fd, dst_fd, st->st_size, src_path); /* duży */
    else
        copy_rw(src_fd, dst_fd, src_path); /* mały */
    close(src_fd); close(dst_fd);
    /* Przywracamy mtime – bez tego demon kopiowałby plik przy */
    /* każdym obudzeniu, bo mtime celu byłby aktualny (teraz) */
    struct utimbuf times = { st->st_atime, st->st_mtime };
    utime(dst_path, &times);
}
```

7. Instrukcja kompilacji i uruchomienia

Kompilacja:

```
make
# lub ręcznie:
gcc -Wall -Wextra -std=c11 -o foldersync main.c daemon.c signals.c sync.c copy.c
```

Składnia wywołań:

```
./foldersync [-R] [-t prog_bajtow] <zrodlo> <cel> [sekundy]
<zrodlo> katalog zrodlowy
<cel> katalog docelowy
[sekundy] interwał synchronizacji (domyslnie: 300)
-R rekurencyjna synchronizacja podkatalogow
-t <bajty> prog rozmiaru dla mmap (domyslnie: 102400 = 100 KB)
```

Przykłady:

```
# Podstawowe uruchomienie (sync co 5 minut)
./foldersync /home/user/docs /backup/docs
# Rekurencyjnie, co 30 sekund, próg mmap = 50 KB
./foldersync -R -t 51200 /home/user/projekt /backup/projekt 30
# Natychmiastowe obudzenie demona
kill -SIGUSR1 $(pgrep foldersync)
# Śledzenie logów
journalctl -t FolderSyncDaemon -f
# Zatrzymanie demona
kill $(pgrep foldersync)
```

Ważna uwaga: katalogi źródłowy i docelowy muszą istnieć przed uruchomieniem demona — program sprawdza to przed daemonizacją i w razie błędu wyświetla komunikat na standardowe wyjście błędów.